

LC 435: Non Overlapping Intervals

Authored by: Arpit Raj, LNMIIT Jaipur | October 1, 2026

Samsung

JPMC

Capital One

o/p → min number of intervals which needs to be removed, so that rest are non overlapping

Data Structure Strategy

basically → let it be

start → 0 → idx

end → 1 → idx

so in a structure we store (pos, type, idx)

then we sort by pos, if pos same sort by type

[classical line sweep]

Algorithm Logic

now we keep a track of currently selected interval if witness an interval which is overlapping we consider the one which finishes first cause it will leave more room for other

and if it is of end type and idx of both current and end are same it means one interval is sorted, update answer and reset curr for up coming intervals

C++ Implementation (Part 1)

```
class Solution {
public:
    int eraseOverlapIntervals(vector<vector<int>>& intervals) {

        int n=intervals.size();
        int sorted=0; //to store how many intervals are there which we will
        actually take

        vector<pair<pair<int,int>,int>> ev;
        for(int i=0;i<n;i++){

            int l=intervals[i][0];
            int r=intervals[i][1];

            ev.push_back({{l,0},i});
            ev.push_back({{r,1},i});
        }

        sort(ev.begin(),ev.end(),[](auto &a,auto &b){
            if(a.first.first==b.first.first){
                return a.first.second>b.first.second;
            }
            return a.first.first<b.first.first;
        });
    }
};
```

C++ Implementation (Continued)

```
int curr=-1;
for(auto e:ev){
    int pos=e.first.first;
    int type=e.first.second;
    int idx=e.second;

    if(type==0){
        if(curr==-1){
            curr=idx;
        }
        else{
            //already koi interval chal rha he jo band bhi nhi hua
            //hence overlaop

            //agar abhi wala jaldi band hoga
            //to wo better choice he
            //as leaves more room
            if(intervals[idx][1]<intervals[curr][1]){
                curr=idx;
            }
        }
    }

    else{
        if(idx==curr){
            sorted++; //ek interval sorted
            curr=-1;
        }
    }
}

return n-sorted;
//jo sort nhi ho paaye wo remove

};
```